

Top-Down Parsing

It is a parsing strategy that first looks at the highest level of the parse tree and works down the parse tree by using the rules of grammar.

Top-down parsing attempts to find the left most derivations for an input string.

In this parsing technique we start parsing from the top (start symbol of parse tree) to down (the leaf node of parse tree) in a top-down manner.

This parsing technique uses Left Most Derivation.

Bottom-Up Parsing

It is a parsing strategy that first looks at the lowest level of the parse tree and works up the parse tree by using the rules of grammar.

Bottom-up parsing can be defined as an attempt to reduce the input string to the start symbol of a grammar.

In this parsing technique we start parsing from the bottom (leaf node of the parse tree) to up (the start symbol of the parse tree) in a bottom-up manner.

This parsing technique uses Right Most Derivation.

The main leftmost decision is to select what production rule to use in order to construct the string.

The main decision is to select when to use a production rule to reduce the string to get the starting symbol.

Example: Recursive Descent parser.

Example: Shift Reduce parser.

<https://www.geeksforgeeks.org/compiler-design/difference-between-top-down-parsing-and-bottom-up-parsing/>

Example with a Context-Free Grammar:

Consider the following simple CFG:

Code

$S \rightarrow E$

$E \rightarrow T + E \mid T$

$T \rightarrow \text{id}$

Let the input string be `id + id`.

Top-Down Parsing (Conceptual):

Start with S .

Apply $S \rightarrow E$. Current goal: derive E from `id + id`.

Apply $E \rightarrow T + E$. Current goal: derive T from `id`, and then E from `+ id`.

Apply $T \rightarrow \text{id}$. Successfully derived `id`. Now, current goal: derive E from `+ id`.

Apply $E \rightarrow T$. This doesn't match `+ id`. Backtrack.

Apply $E \rightarrow T + E$. (This is an example of potential backtracking in some top-down parsers).

Apply $T \rightarrow \text{id}$. Successfully derived `id`. Now, current goal: derive E from `id`.

Apply $E \rightarrow T$.

Apply $T \rightarrow \text{id}$. Successfully derived `id`.

The parse tree is built by expanding non-terminals until the input terminals are matched.

Bottom-Up Parsing (Conceptual):

Start with the input string `id + id`.

Recognize `id` as T using $T \rightarrow \text{id}$. String becomes `T + id`.

Recognize `id` as T using $T \rightarrow \text{id}$. String becomes `T + T`.

Recognize T as E using $E \rightarrow T$. String becomes `E + T`.

Recognize `E + T` as E using $E \rightarrow E + T$. String becomes `E`.

Recognize E as S using $S \rightarrow E$. String becomes `S`.

The input string has been reduced to the start symbol, and the parse tree is constructed by combining subtrees as reductions occur.